# Computer Organization and Assembly Language

## Week 04 — Chapter 2: Addressing Modes
## Data Declaration and Direct Addressing

Dr. Maqsood M. Khan

Department of Computer Science / Software Engineering / Artificial Intelligence
FAST-NUCES Peshawar

Week 04

# Outline


1. Quick Review
2. Why Addressing Modes?
3. Data Declaration
4. Direct Addressing
5. One Label, Many Variables
6. Memory as the Destination
7. Legal and Illegal Operand Combinations

# Quick Review: What Was Done Last Week?

- The `.lst` file: **line, address, opcode, source**; addresses are a running byte count, $\text{addr}_{n+1} = \text{addr}_n + \text{length}_n$.
- Instruction encoding: `B8 05 00` = opcode + 2-byte immediate; `01 D8` = opcode + ModRM (register selector byte).
- **iAPX 88 (8088)**: 16-bit processor, 20-bit address bus, **1 MB** of memory versus 64 KB for the 8085.
- **14 registers**: AX BX CX DX | SI DI | IP SP BP | Flags | CS DS SS ES.
- **Segmented memory model**: four movable 64 KB windows.

$$\text{Physical Address} = (\text{Segment} \times 16) + \text{Offset}$$

- Paragraph boundaries, overlapping segments, logical vs physical addresses.
- `org 0x0100` — a `.COM` program starts at offset `0100h` because of the **PSP** (Program Segment Prefix).

# What Is This All About?

## The question of this chapter

An instruction needs operands. **Where do those operands come from?**
The different answers to that question are called addressing modes.

**Immediate**
value inside
the instruction

```
mov ax, 5
```

**Register**
value inside
the CPU

```
add ax, bx
```

**Direct**
value inside
memory

```
mov ax, [num1]
```

- The first two we already know. **This week we add the third**: data kept in memory.

# Why Do We Need to Understand This?

- **Because real programs have variables.** Constants alone cannot express anything useful — a program that cannot store a changing value cannot loop, count, or keep a result.
- **It explains high-level languages.** An `int x` in C++ is nothing more than a labelled memory location. After this chapter you will know exactly what the compiler does to your variables.
- **It explains pointers.** Square brackets in assembly are the same idea as the $*$ operator in C.
- **It makes the listing file readable.** You will be able to look at an opcode and say which operand came from memory and which from a register.
- **It prepares arrays and strings.** Indexed and based addressing (next chapters) are built directly on top of what we do today.

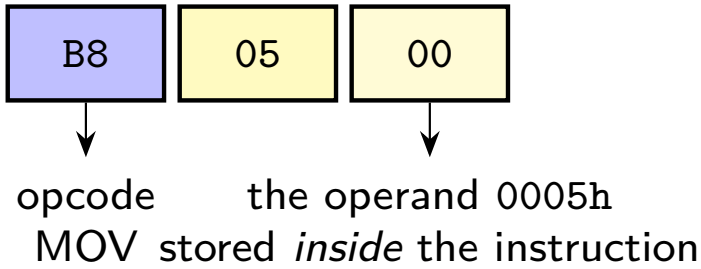
**By the end of this lecture you should be able to**

declare data with `db`/`dw`, use labels, read and write memory variables, and explain *why* some instructions are illegal.

# Where We Left Off: the Immediate Operand

- Our very first instruction was:

```
mov ax, 5
```




- `MOV` = opcode, `AX` = destination operand, 5 = source operand.
- The value 5 is stored as **part of the instruction encoding**. Such an operand is called an immediate operand.
- It is only *one* of many operand types available.

# Why Immediate Operands Are Not Enough

- Writing programs using only immediate operands is difficult.
- Every reasonable program needs **data in memory**, apart from a few constants.
- A constant cannot be changed — so it can never be the **destination** operand.

```
mov 5, ax
```

meaningless *and* illegal — you cannot store into a number

- Only **registers** or **data placed in memory** can be changed.
- So the real data of a program lives in memory, with only a few constants.
- $\Rightarrow$ we need a mechanism to **store and retrieve data from memory**.

# Declaring Data: db and dw

To mark a part of our program as **data** instead of **instructions**, we use special *assembler directives*:

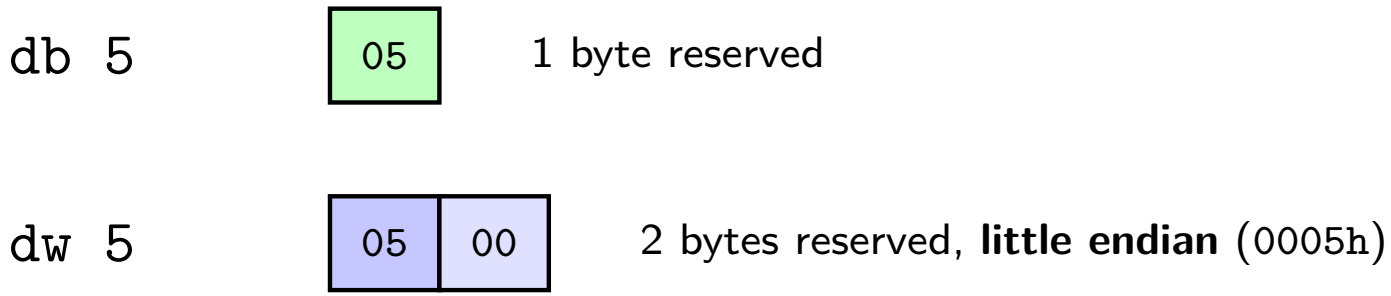
```
db somevalue ; define byte -> reserves 1 byte (8 bits)
dw somevalue ; define word -> reserves 2 bytes (16 bits)
```

- A cell in memory is reserved, containing the desired value.
- Now we can have **variables** instead of constants.
- Directives also exist for double words and quad words, but we restrict ourselves to `db` and `dw` for now.

## Directive vs Instruction

`mov`, `add` are **instructions** — they become opcodes the CPU executes. `db`, `dw`, `org` are **directives** — orders to the *assembler*, executed at assembly time.

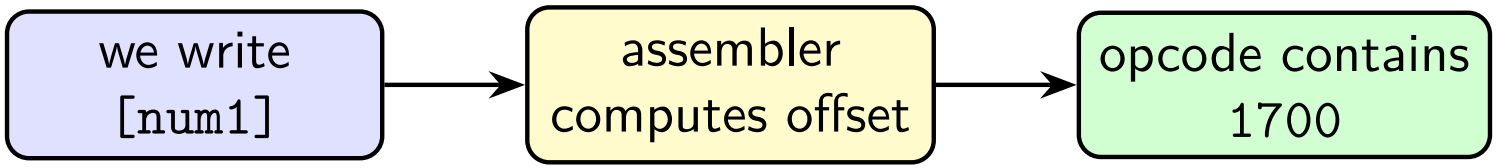
# db vs dw — What Really Happens in Memory




- `dw 5` places `05 00` in memory — read back as a word it is `0005h`.
- `db 5` would place just `05` in one byte.
- This is why consecutive `dw` variables are 2 apart, not 1 apart — remember this for the next examples.

# Labels: Giving a Name to an Address

- To use a variable later we need **its address**. The assembler helps us: we associate a **symbol** with an address and use that symbol in the rest of the code.
- Such a symbol is called a label, written as the name followed by a colon: `num1:  dw 5`
- The assembler computes the address from the `org` directive plus the lengths of all instructions and data in between, then replaces every reference with that number.




- Exactly like variables in a high-level language — the compiler turns names into addresses; here the process is just **one level less hidden**.

# Example 2.1 — Adding Three Numbers from Memory

```
; a program to add three numbers using memory variables
[org 0x0100]
; mov [num1], [num2] ; ILLEGAL
    mov ax, [num1] ; load first number in ax
    mov bx, [num2]
    add ax, bx
    mov bx, [num3]
    add ax, bx
    mov [num4], ax ; store the result
    mov ax, 0x4c00
    int 0x21

num1: dw 5
num2: dw 10
num3: dw 15
num4: dw 0
; watch the listing carefully
```

- The numbers 5, 10 and 15 are now **memory variables**, not constants.
- **Square brackets** mean "the *contents of* this address".

# Example 2.1 — The Listing File

```
 1
 2 [org 0x0100]
 3 00000000 A1[1700] mov ax, [num1]
 4 00000003 8B1E[1900] mov bx, [num2]
 5 00000007 01D8 add ax, bx
 6 00000009 8B1E[1B00] mov bx, [num3]
 7 0000000D 01D8 add ax, bx
 8 0000000F A3[1D00] mov [num4], ax
 9
10 00000012 B8004C mov ax, 0x4c00
11 00000015 CD21 int 0x21
12
13 00000017 0500 num1: dw 5
14 00000019 0A00 num2: dw 10
15 0000001B 0F00 num3: dw 15
16 0000001D 0000 num4: dw 0
```

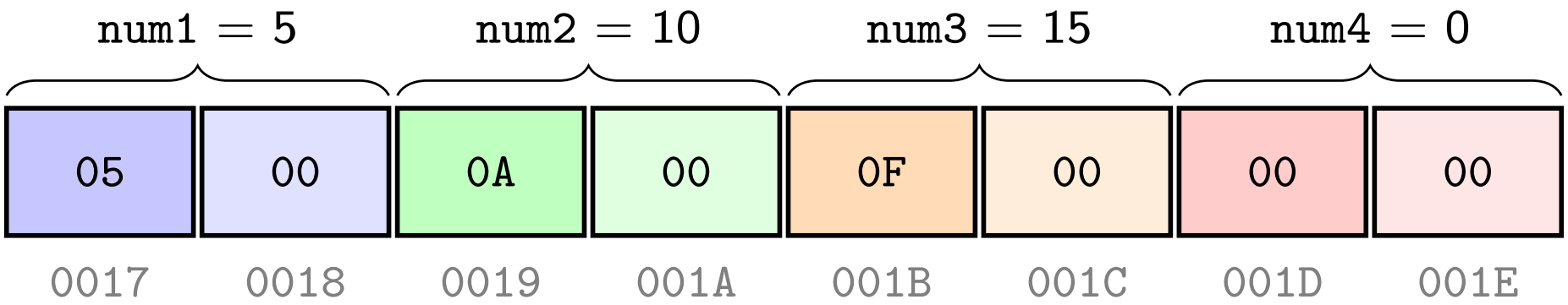
- Note how the code ends at 0017 — exactly where num1 begins.

# The Opcode Has Changed: B8 → A1

| | | | | |
|---|---|---|---|---|
| `mov ax, 5` | B8 | 05 | 00 | **constant** into AX — operand *is* the value |
| `mov ax, [num1]` | A1 | 17 | 00 | **memory** into AX — operand is the *address* |

- `B8` moves a **constant** into AX; `A1` moves **data from memory** into AX.
- The operand `1700` read as a word is `0017h` = **23 decimal** — and the bottom of the listing shows that this is exactly the offset of `num1`.
- The assembler calculated the offset of `num1` and substituted it everywhere in the program.
- Similarly `num2`, `num3`, `num4` live at `0019`, `001B`, `001D`.

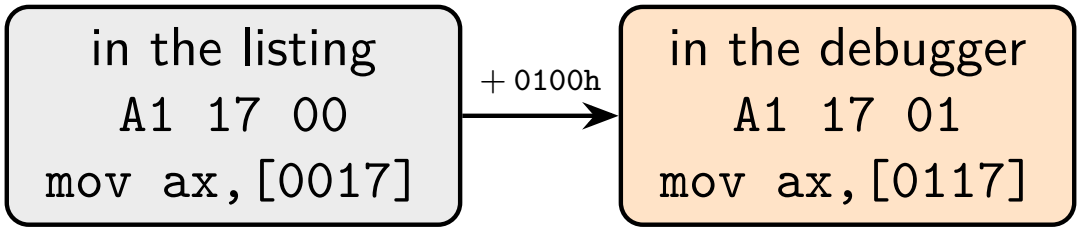
# The Data Area in Memory




- Each `dw` takes **2 bytes**, so the labels are `0017`, `0019`, `001B`, `001D` — each 2 apart.
- The value `0500` at offset `0017` is read back as the word `0005h`: the contents of location `0017` are 5.

# Assembly Time vs Load Time

- The listing counts from 0000, but the program is **loaded at offset** 0100 (the PSP occupies the first 256 bytes).
- So every memory access in our program is **displaced by** 0100h.




$$0017\text{h} + 0100\text{h} = 0117\text{h}$$

- In the debugger's data window, offset 0117 indeed contains 0005.
- Step through the program and watch: memory is read, registers update, IP moves forward, and the result is saved back to memory.

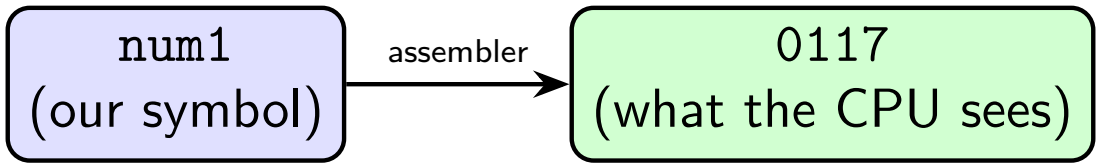
# Two Warnings from the Debugger

## 1. The debugger shows your data as code

Below the termination instruction you will see meaningless "instructions". The debugger disassembles **everything** in the code window and cannot tell your declared data from opcodes. It is your responsibility to terminate execution before the data is reached and executed as code.

## 2. Your names are gone

The debugger shows `0117`, `0119`, `011B`, `011D` — not `num1`...`num4`. Our numerical machine only works with numbers; labels were for *our* ease and the assembler converted them automatically.



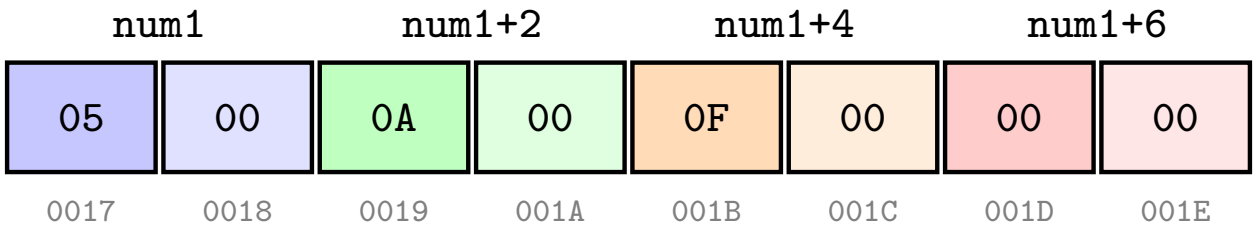
# Example 2.2 — Using a Single Label

```
; add three numbers accessed using a single label
[org 0x0100]
    mov ax, [num1]
    mov bx, [num1 + 2] ; notice how we can do arithmetic here
    add ax, bx ; also, why +2 and not +1 ?
    mov bx, [num1 + 4]
    add ax, bx
    mov [num1 + 6], ax ; store the sum at num1+6
    mov ax, 0x4c00
    int 0x21

num1: dw 5
      dw 10
      dw 15
      dw 0
```

- Every location is now accessed **relative to** `num1`. Only one label is needed.

# Why +2 and Not +1?




- Addresses count **bytes**, but each `dw` variable occupies **two** bytes.
- `num1+1` would point at the *high half* of the first number — not at the second variable.
- `num1+2` is a constant expression: no variables are involved, so the assembler evaluates it at assembly time:

$$0117\text{h} + 2 = 0119\text{h}$$

- Inside the debugger this program is a **verbatim copy** of the previous one — the assembler catered for the difference.

# Example 2.3 — Same Data, Shorter Declaration

```
; add three numbers accessed using a single label
[org 0x0100]
    mov ax, [num1]
    mov bx, [num1 + 2]
    add ax, bx
    mov bx, [num1 + 4]
    add ax, bx
    mov [num1 + 6], ax
    mov ax, 0x4c00
    int 0x21

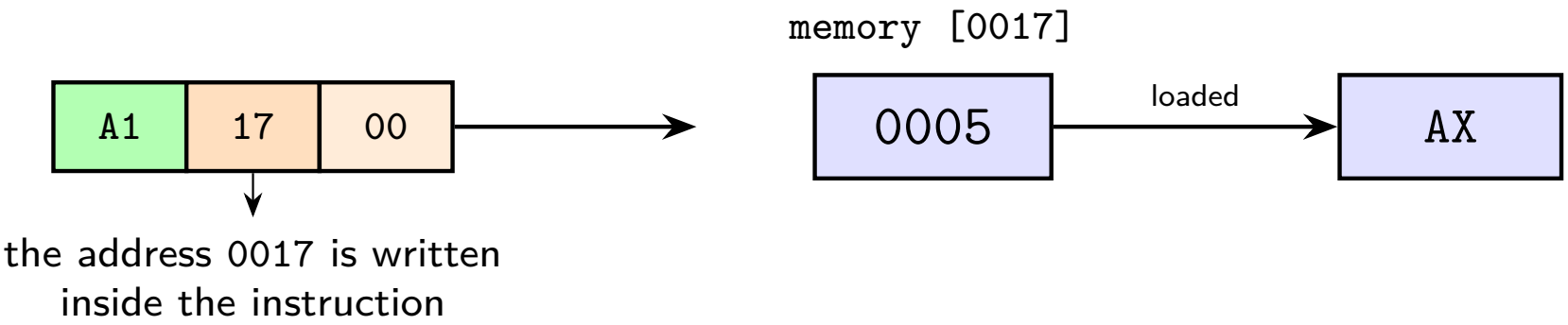
num1: dw 5, 10, 15, 0 ; four words, one line
```

- One `dw` can define several values separated by commas.
- The bytes produced are `05 00 0A 00 0F 00 00 00` — exactly the same as Example 2.1 and 2.2.
- Three different ways of writing it, **one identical program** in memory.

# What Is Direct Addressing?

## Definition

In **direct addressing** the memory address is fixed and is given inside the instruction.




- The data now lives in memory, so it can also be used as the **destination** operand.
- Source and destination must have the **same size**: a word variable is read into a **word** register.
- The bytes 05 00 in memory become the correct word 0005 inside the register — registers hold data in proper order.

# Example 2.4 — Adding Directly Into Memory

```
; a program to add three numbers directly in memory
[org 0x0100]
    mov ax, [num1] ; load first number in ax
    mov [num1+6], ax ; store first number in result
    mov ax, [num1+2] ; load second number in ax
    add [num1+6], ax ; add second number to result
    mov ax, [num1+4] ; load third number in ax
    add [num1+6], ax ; add third number to result
    mov ax, 0x4c00 ; terminate program
    int 0x21

num1: dw 5, 10, 15, 0
```

- New idea: we can add a **register into a memory variable** — the destination of the addition is now **memory**.

# Example 2.4 — The Listing File

```
 1
 2 [org 0x0100]
 3 00000000 A1[1900] mov ax, [num1]
 4 00000003 A3[1F00] mov [num1+6], ax
 5 00000006 A1[1B00] mov ax, [num1+2]
 6 00000009 0106[1F00] add [num1+6], ax
 7 0000000D A1[1D00] mov ax, [num1+4]
 8 00000010 0106[1F00] add [num1+6], ax
 9
10 00000014 B8004C mov ax, 0x4c00
11 00000017 CD21 int 0x21
12
13 00000019 05000A000F000000 num1: dw 5, 10, 15, 0
```

- num1 now sits at 0019 (the code became longer) — handled **automatically** by the assembler because we used labels instead of hard-coded addresses.

# Comparing the Two add Encodings

| Instruction | Opcode | Bytes | Destination |
|---|---|---|---|
| `add ax, bx` | `01 D8` | 2 | register |
| `add [num1+6], ax` | `01 06 1F 00` | 4 | memory |

- Same opcode `01` (ADD), but the **ModRM byte changed** from `D8` to `06`:
  - `D8` $\rightarrow$ destination is a *register*.
  - `06` $\rightarrow$ destination is a *memory address*, which then follows as `1F 00`.
- That is why the instruction grew from 2 bytes to 4 bytes.
- `num1+6` = `0019h` + 6 = `001Fh` — computed by the assembler.
- The addressing method is **direct addressing** in both `MOV` and `ADD`.

# Why Not Memory to Memory?

The first two instructions of the last program read a number into AX and put it somewhere else. A natural thought:

```
mov [num1+6], [num1] ; ILLEGAL
```

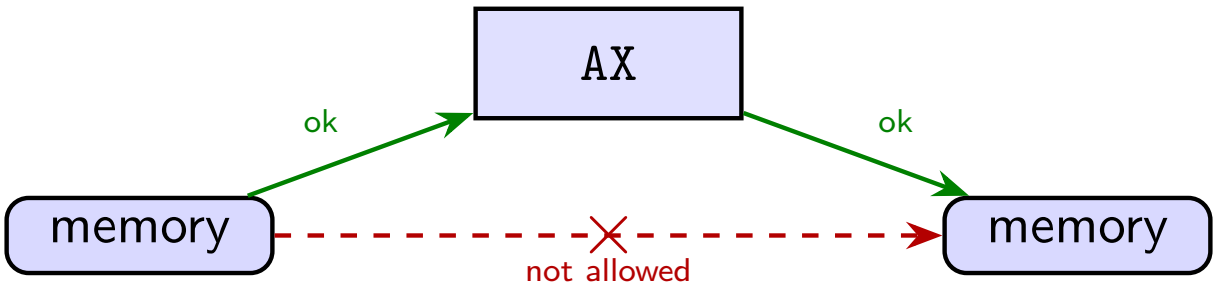
- This form is **not allowed** on the Intel architecture.
- None of the general operations (`mov`, `add`, `sub`, ...) can move data from memory to memory.
- Only **string instructions** can do memory-to-memory movement — discussed later in the course.

## Rule of thumb

One instruction can have **at most one operand in brackets**, otherwise the assembler gives an error.

# Allowed and Disallowed Combinations

| Destination | Source | Allowed? |
|---|---|---|
| register | register | ✓ |
| register | memory | ✓ |
| memory | register | ✓ |
| register | constant | ✓ |
| memory | constant | ✓ |
| memory | memory | ✗ |
| constant | anything | ✗ |




- Memory to memory must always go **through a register**.

# Summary

- An **immediate operand** is stored inside the instruction encoding; a constant can never be a destination.
- `db` reserves **1 byte**, `dw` reserves **2 bytes**. `dw 5` → `05 00` (little endian).
- A **label** is a name for an address; the assembler replaces it with a number (`num1` → `0017`).
- **Direct addressing**: the fixed address is written inside the instruction, e.g. `A1 17 00` = `mov ax,[num1]`.
- Opcode tells you where the operand comes from: `B8` = constant into AX, `A1` = memory into AX.
- At load time everything shifts by `0100h`: `0017` → `0117`.
- Word variables are **2 apart**: `num1+2`, `+4`, `+6` — constant expressions evaluated at assembly time.
- Memory can be a **destination** (`add [num1+6], ax`), but memory to memory is illegal — at most one bracketed operand per instruction.

# Thank You

Any Questions?

*"A variable is nothing but an address with a name. Once you can see the address behind the name, you can read any program ever written."*

Next: Register indirect addressing, size mismatch errors, register + offset addressing, segment association, and address wraparound .